Will It Catch Tomorrow's Attack? How Operators Should Evaluate Learned Network Intrusion Detectors Before Trusting Them on Zero-Day Traffic

Hafsa [surname]*, [co-authors]
*[Department, University, City, Country]
Email: [email]

***Abstract*—Operators deploy learned network intrusion detection systems (NIDS) to catch attacks absent from their training data, yet detectors are validated by holding one attack class out of a randomly split benchmark (LOACO). We ask whether this laboratory protocol predicts behaviour when a new attack class appears later in time, alongside drifting benign traffic. We build a pre-registered, leakage-controlled benchmark on four NetFlow-v3 datasets and LYCOS-IDS2017, with time-block splits and deduplicated flow windows, and evaluate 12 detectors from five families over five seeds. Three findings matter for operators. First, LOACO is often infeasible on time-ordered captures: temporal splits yield zero hold-out folds on two of three day-scheduled datasets, while natural temporal novelty remains measurable. Second, the protocols disagree: the best detector family changes with dataset and protocol, and a plain XGBoost classifier matches or beats the best novelty detector in four of seven evaluations. Third, natural novelty raises the seed variability of every supervised score 1.7–17-fold, and max-softmax and energy scores fall below chance because confidently detected attacks read as normal. We derive an evaluation checklist for detector assurance and release all splits, code and pre-registration records.**

***Index Terms*—network security management, intrusion detection, zero-day attacks, service assurance, evaluation methodology, NetFlow**

I. Introduction

Anetwork operator who deploys a learned network intrusion detection system (NIDS) is buying protection against attacks that are not in its training data. Before trusting the detector, the operator needs evidence that it raises an alarm when a new attack class first appears in the network. This is a security-management decision with operational cost: a detector that misses novel attacks creates false assurance, and one whose behaviour changes from one retraining to the next cannot be operated predictably.

The problem. The evidence operators receive comes from a laboratory protocol, leave-one-attack-class-out (LOACO): one attack class is removed from a randomly split benchmark and the detector must flag it at test time. Recent zero-day NIDS work, including contrastive training [22] and transformer-based open-set detection [23], reports results under this protocol. LOACO, however, differs from deployment in three ways. It removes a class artificially while keeping benign traffic from the same hours as the test set; it ignores time, although new attacks arrive after training together with new hosts, tools and benign drift; and it is usually reported for one training run. Security-ML research has repeatedly shown that such gaps inflate results [1], [2], [3], [4].

The gaps. We identify three gaps in current NIDS evaluation. (G-1) Zero-day NIDS studies use either class hold-out [20], [21] or time-aware splits [25], [15]; a few recent single-method papers apply both [18], [19], but no study compares several detector families under class hold-out and *natural temporal novelty*, in which attack classes appear only after the training period, on the same data. It is therefore unknown whether LOACO rankings predict deployment. (G-2) NIDS papers score novelty with classifier-confidence scores designed for multi-class image recognition, such as maximum softmax probability (MSP) [37] and energy [38]. These have been reported to perform poorly, even below chance, on unseen attacks in multi-class NIDS [13], [14], but why, and whether they are valid at all for the common two-class detector, has not been characterised. (G-3) Seed variability is rarely reported per class and hypotheses are rarely pre-registered [46], so selective reporting cannot be ruled out.

Research questions. We ask: **RQ1** Is LOACO feasible on realistic, time-ordered captures, and is natural novelty an available alternative? **RQ2** Do LOACO and natural novelty agree on which detector family is best, and how stable are the scores across training seeds? **RQ3** Which novelty scores are valid for binary NIDS, and how do temporal and cross-dataset shifts affect supervised detectors?

Approach and novelty. We build a pre-registered, leakage-controlled benchmark on five public datasets: four NetFlow-v3 datasets [31] and LYCOS-IDS2017 [8]. Flows are grouped into time blocks before windowing, windows are deduplicated by hash, and every split is checked fail-closed. We evaluate 12 detectors in five families (classifier-confidence, representation-distance, benign-only, contrastive and a supervised XGBoost reference) under LOACO, natural temporal novelty, a purged time-block hold-out, in-distribution splits and zero-shot cross-dataset transfer, each over five seeds. Unlike prior work, which evaluates zero-day detection either by class hold-out

or by time-aware splits, or applies both to a single proposed method [18], [19], we compare five detector families under both protocols on the same data. Whereas prior work reports aggregate run-to-run variance [14], we report per-class seed variability for every detector. To our knowledge, this is also the first pre-registered NIDS benchmark, and we report its failed hypotheses. Every number comes from one code commit and is regenerated from stored results.

Contributions.

- **C1 Protocol feasibility (RQ1).** Attacks in day-scheduled captures are confined to particular days, which has made chronological splits look degenerate [16]. We quantify the consequence: temporal LOACO yields zero feasible folds on CIC-IDS2018 and ToN-IoT and one on LYCOS-IDS2017 under a purged block split, while the same property provides 2–5 natural-novelty classes per dataset, which we use as a zero-day protocol.
- **C2 No family dominates (RQ2).** The best family changes with dataset and protocol, and, in contrast to studies that favour unsupervised detectors on unknown attacks [21], [17], XGBoost matches or beats the leading novelty family in four of seven evaluations.
- **C3 Hidden instability (RQ2).** Natural novelty raises the seed standard deviation of every supervised score 1.7–17$\times$ over LOACO, while seeded benign-only detectors stay stable.
- **C4 Score validity (RQ3).** We show why MSP and energy fail in binary NIDS: a confidently detected novel attack reads as in-distribution, so both fall below chance, while the classifier's attack probability does not on average.
- **C5 Generalization (RQ3).** Temporal-split effects are dataset-specific, and 67% of 108 zero-shot cross-dataset results fall between 0.3 and 0.6 AUC.
- **C6 Negative results and release.** Four pre-registered hypotheses failed their frozen gates; we report them and release splits, code and amendments.

From these we derive a six-point checklist that operators can use to assure a detection service before deployment (Section VII). Section II gives background, Section III reviews related work, Section IV describes the benchmark, Section V the experiments, Section VI the results and Section VII the discussion.

II. Background

A. Flow-based NIDS and windowing

Flow-based NIDS summarize each connection as a record of counters and timings exported by network monitoring infrastructure. Sarhan et al. converted several public captures into a common NetFlow feature set so that detectors can be compared across networks [32]; the third version (v3) adds temporal features and per-flow start and end timestamps [31], which make strictly time-ordered splits possible. Sequence detectors consume windows of T consecutive flows. Windowing creates two leakage risks: overlapping windows can place near-identical samples in training and test, and flows from one attack episode can straddle the split. Grouping flows into contiguous time blocks before windowing and deduplicating windows by hash remove both.

B. Zero-day evaluation protocols

Three protocols are compared in this paper (Fig. 1, stage 4). *LOACO* removes one attack class from the training pool; the test set holds benign windows plus that class, and splits are random, here grouped by 1-hour blocks. *Natural temporal novelty* splits flows in time; attack classes that occur only after the training period are unseen by construction, and benign traffic drifts as it would in operation. *Purged block LOACO*, for short captures, assigns short time blocks to splits within each day and drops blocks adjacent to test or validation blocks, following purged cross-validation for time series [45]. LOACO asks whether a detector generalizes to a missing class under the training distribution; natural novelty asks the operator's question, under real temporal shift.

C. Novelty scores and the binary pitfall

Out-of-distribution (OOD) detection assigns each input a score that should be higher for inputs unlike the training data. *Classifier-confidence* scores read the output layer: MSP scores $1 - \max_k p_k$ [37] and energy scores $-\log \sum_k e^{f_k}$ over logits f_k [38]. *Representation-distance* scores measure how far hidden features lie from benign training features, e.g., Mahalanobis [39] or nearest-neighbour distance [40]. *Benign-only* detectors never see attacks [41], [42]. *Contrastive* detectors such as CLAD [22] learn an embedding from benign and known attacks. MSP and energy measure how sure a classifier is, not which class it chose. In a two-class NIDS, a new attack that is confidently classified as *attack* receives a low MSP score and therefore reads as in-distribution: the expected outcome of successful detection becomes a missed novelty. In a binary detector this is not an edge case but the expected outcome whenever a novel attack is detected.

III. Related Work

Evaluation pitfalls and dataset quality. Sommer and Paxson argued that anomaly detection outside a closed world is harder than benchmark scores suggest [1]. Arp et al. catalogued ten pitfalls in security ML, including sampling bias and data snooping [2], and TESSERACT showed that ignoring time inflates malware-detection results [3]. For NIDS, Apruzzese et al. found that evaluations rarely reflect operational conditions [4], and Husák et al. reported, from a network-management perspective, that ML intrusion detection research favours ML metrics over operational outcomes [5]. Dataset audits found labelling and flow-construction errors in CIC-IDS2017/2018 [6], [7], corrected in LYCOS-IDS2017 [8]; design smells in benchmark NIDS datasets that cause severe experimental bias [9]; and metadata features that let models shortcut the task [10]. These works diagnose problems; we measure how one of them, the choice of zero-day protocol, changes which detector an operator would deploy.

Zero-day and open-set NIDS. A recent survey in IEEE TNSM concludes that ML intrusion detectors detect known attacks well but unknown, atypical and polymorphic attacks poorly [11]. Sarhan et al. framed zero-day detection as zero-shot learning and linked low detection rates to large Wasserstein distance from known attacks [20]. Zoppi et al. compared 47 algorithms on unknown attacks and found unsupervised meta-learners strongest [21]. Wilkie et al. proposed CLAD, a contrastive loss trained on benign and known attacks [22]; Jodayree et al. combined a transformer with energy scoring and extreme-value calibration, reporting an AUROC of 0.975 under leave-one-attack-out on IoMT traffic [23]; Thi and Gurusamy detect open-set attacks with a model trained on benign traffic only [24]; and Baye et al. benchmarked deep open-set recognition for NIDS at NOMS [12]. Softmax-confidence baselines have scored below chance on held-out attacks in multi-class NIDS [13], and softmax entropy and energy trailed uncertainty-aware models [14]; these observations were made for multi-class detectors and without an explanation tied to the binary setting. All of these works evaluate under class hold-out or in-distribution.

Drift and temporal evaluation of NIDS. INSOMNIA applied time-aware evaluation to intrusion detection and found that accounting for drift is vital [25]; ReCDA [26] and continual-learning NIDS [27] adapt detectors to drifting traffic. Recent work examines temporal protocols directly: Abdessamad and Tazi found that temporal splits on ToN-IoT leave unseen attack classes in test and that classifier rankings depend on the protocol [15]; Moczkodan and Ragab found a chronological CIC-IDS2017 split degenerate because attacks were scheduled by day [16]; Mesca et al. compared classifiers under random, attack-held-out and leave-one-out protocols on industrial traffic [17]; and ThreatFormer-IDS [18] and Alhalangy and Eltayeb [19] evaluate a single proposed detector under both chronological and held-out-family splits. None compares detector families under class hold-out versus natural temporal novelty, the question an operator faces when choosing a detector.

Cross-domain generalization. NetFlow NIDS are commonly evaluated in-distribution, e.g., the graph-based E-GraphSAGE [28]. Layeghy and Portmann [29] and Cantone et al. [30] found that near-perfect within-dataset performance falls towards chance across datasets; These measure transfer between networks but not novelty within one network.

Positioning. Table I summarises the gap: prior work evaluates either class hold-out or cross-dataset transfer, typically for one detector family, without per-class seed variability or pre-registration.

TABLE I
POSITIONING AGAINST REPRESENTATIVE PRIOR WORK (–: NOT ASSESSED).

Work	Zero-day protocol	X-data	Families	Seed var.	Pre-reg.
Sarhan et al. [20]	zero-shot hold-out	no	supervised	–	no
Zoppi et al. [21]	unknown attacks	no	sup./unsup./meta	–	no
Layeghy & Portmann [29]	–	yes	supervised	–	no
Wilkie et al. [22]	LOACO	no	contrastive, anomaly	–	no
Jodayree et al. [23]	leave-one-out	1 pair	energy + EVT	5 seeds	no
This work	**LOACO + natural + purged**	**9 pairs**	**5 (12 det.)**	**per class**	**yes**

IV. BENCHMARK METHODOLOGY

Fig. 1 shows the benchmark as four stages: the input datasets, a leakage-controlled processing pipeline, the backbone and its 12 detectors, and the evaluation protocols that produce the outputs C1–C6. Data, splits, backbone and detectors were fixed in advance, and every protocol change was recorded as a dated, hashed amendment before the runs it governs (Section IV-E).

A. Stage 1: Input datasets

All five datasets are public, were downloaded from their official sources and are recorded in a manifest with SHA-256, row count and column list (Table II). D1–D4 are the NetFlow-v3 releases of the University of Queensland [31], which re-extract each capture's packet traces into 53 NetFlow features with per-flow start and end timestamps in milliseconds, using a common feature set across datasets [32].

- **D1 CSE-CIC-IDS2018** [33] was generated by the Canadian Institute for Cybersecurity on a cloud-hosted enterprise testbed with scripted benign users and attack machines; our copy spans 14 Feb–2 Mar 2018. Its 14 attack labels (brute force, botnet, DoS, DDoS, web attacks, infiltration) map onto six families. Labelling and feature-generation errors in the original release have been documented [7].
- **D2 UNSW-NB15** [34] was created in the UNSW Canberra Cyber Range with a traffic generator that mixes real normal activity with nine synthetic attack families (e.g., exploits, fuzzers, reconnaissance, backdoors); it spans 22 Jan–18 Feb 2015.
- **D3 ToN-IoT** [35] comes from an IoT/IIoT testbed with edge, fog and cloud layers and nine attack types (scanning, DoS, DDoS, ransomware, backdoor, injection, cross-site scripting, password and man-in-the-middle); it spans 23–29 Apr 2019.
- **D4 BoT-IoT** [36] simulates IoT devices under botnet attacks (DDoS, DoS, reconnaissance and data theft); it spans May–Jun 2018 but contains only 1,315 benign windows, so it serves only as a transfer target.
- **D5 LYCOS-IDS2017** [8] re-processes the CIC-IDS2017 packet traces (five working days, 3–7 Jul 2017) with a corrected flow extractor and relabelling that fix documented flow-construction and labelling errors [6], [7]. We ran the authors' labelling script unmodified, kept its 73 own features and do not use D5 for transfer.

B. Stage 2: Leakage-controlled processing

Each step addresses a known source of optimistic bias [2], [9], [10].

1) *Audit and drop.* We record rows, types, missing values and class counts, then drop identifier columns (addresses, ports, DNS query ID), whose values iden-

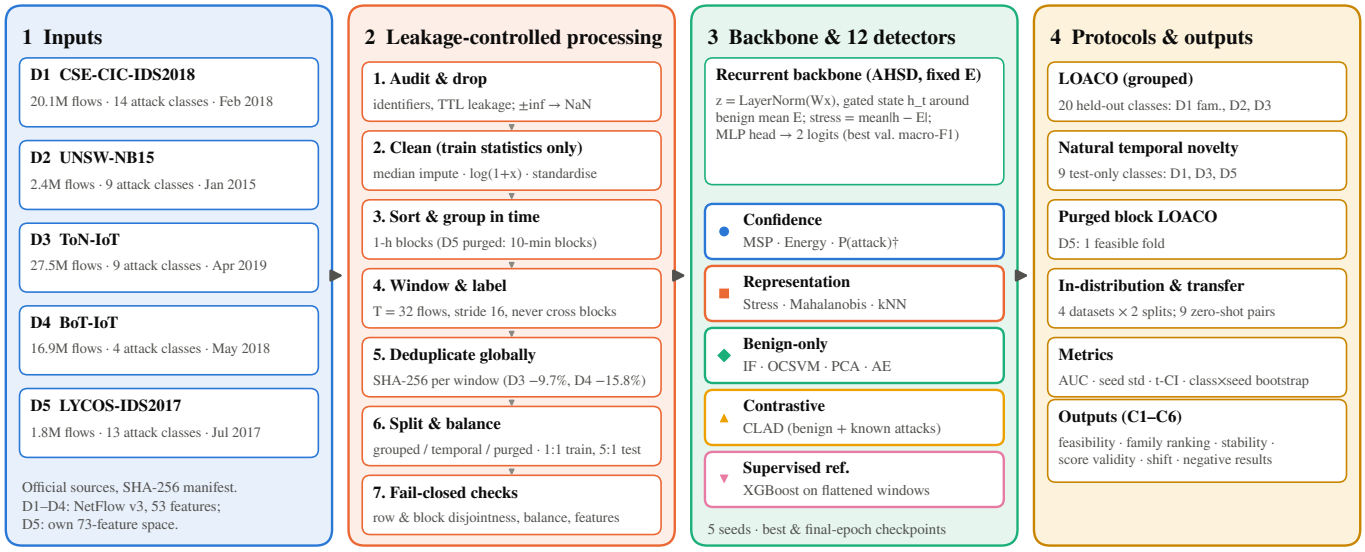


Fig. 1. Benchmark methodology. (1) Five public datasets enter with checksummed manifests. (2) Seven processing steps turn flows into deduplicated, time-blocked windows and fail-closed checked splits. (3) One recurrent backbone supplies the classifier-confidence and representation-distance scores; benign-only, contrastive and XGBoost detectors are trained on the same splits. (4) Five evaluation settings, each over five seeds, produce the six results C1–C6.

TABLE II
DATASETS. WINDOWS ARE COUNTED AFTER GLOBAL DEDUPLICATION.

ID	Dataset	Flows	Windows	Att. cl.	Role
D1	NF-CSE-CIC-IDS2018-v3	20,115,529	1,255,180	14	LOACO, NN, transfer
D2	NF-UNSW-NB15-v3	2,365,424	147,801	9	LOACO, transfer
D3	NF-ToN-IoT-v3	27,520,260	1,553,191	9	LOACO, NN, transfer
D4	NF-BoT-IoT-v3	16,933,808	890,852	4	transfer target
D5	LYCOS-IDS2017	1,837,498	114,783	13	NN, purged LOACO

tify hosts rather than behaviour [10], and minimum/maximum TTL, a known label-leakage artifact. Timestamps are kept only for ordering and splitting.

2) *Clean with training statistics only.* Infinite values become missing and are imputed with the training median; non-negative heavy-tailed counts are transformed with $\log(1+x)$; features are standardized with training mean and deviation; zero-variance features and one of each pair with $|r| \geq 0.999999$ are removed. For D1–D4 the intersection leaves 41 common features. No statistic is ever fitted on test or transfer-target data.
3) *Sort and group in time.* Flows are sorted by start time and grouped into contiguous 1-hour blocks (10-minute blocks for D5's purged split), so that flows of one attack episode stay together.
4) *Window and label.* Windows of T=32 consecutive flows with stride 16 are cut inside blocks and never cross a block boundary. A window is an attack if any flow is an attack, and its class is the majority attack class.
5) *Deduplicate globally.* Every window is hashed over its raw features and exact duplicates are removed across the whole dataset, so that no window appears in both training and test (9.7% of D3 and 15.8% of D4 windows were duplicates).
6) *Split and balance.* Blocks, never individual flows or windows, are assigned to splits (Section IV-D). Training sets are balanced 1:1 (7,000 benign and 7,000 attack windows); validation and test sets use a 5:1 benign-to-attack ratio, capped at 2,500 benign and 500 attack windows.
7) *Fail-closed checks.* The pipeline stops if any check fails: row and block disjointness across splits, deduplication, balance, class composition and feature lists. 538 checks ran at preprocessing for D1–D4 and further checks for every derived split (e.g., 32 for D5's purged split); all passed.

C. Stage 3: Backbone and detectors

Backbone. All supervised scores are read from one recurrent backbone, the fixed-equilibrium variant of a homeostatic sequence detector. Each flow vector is embedded as $z_t = \mathrm{LayerNorm}(Wx_t) \in \mathbb{R}^{128}$. A state h_t, initialised at a benign reference E, tracks the input through per-step, per-channel gates:

$$h_t = h_{t-1} + \alpha_t \odot (z_t - E) - \beta_t \odot (h_{t-1} - E),$$
$$[\alpha_t, \beta_t] = \sigma\big(W_g[z_t, h_{t-1}]\big).$$

E is not learned: it is the mean embedding of benign training windows, tracked by an exponential moving average during training and set exactly afterwards, and it stays fixed within a window. Homeostatic *stress* is $\frac{1}{TD}\sum_{t,d}|h_{t,d} - E_d|$. A two-layer MLP head (linear–GELU–linear) reads $[h_T,\ \frac{1}{T}\sum_t h_t,\ \text{stress}]$ and outputs two logits; its hidden layer is the penultimate representation. We chose this backbone because it exposes a representation-distance score inside the model.

Detectors. Twelve detectors in five families score each test window; higher scores always mean more anomalous, and orientation was fixed before any run.

- *Classifier-confidence* (from the backbone's logits): MSP, energy, and P(attack), the softmax attack probability, which was added after the last confirmatory gate and is marked †.
- *Representation-distance* (from the backbone's states): stress; Mahalanobis distance to a Ledoit–Wolf Gaussian fitted on benign penultimate features [39]; and distance to the 10th benign nearest neighbour of ℓ_2-normalised penultimate features [40].
- *Benign-only* (fitted on benign training windows, never seeing attacks): Isolation Forest with 200 trees [41], one-class SVM with RBF kernel and ν=0.1 [42], PCA reconstruction error at 95% retained variance, and a fully connected autoencoder.
- *Contrastive*: CLAD [22], trained on benign and known attacks and implemented from its paper, with the details it leaves open fixed before training.
- *Supervised reference*: XGBoost [43] on flattened windows (300 trees, depth 6, learning rate 0.1), scored by its attack probability.

D. Stage 4: Evaluation protocols

grouped_random shuffles blocks into 60/20/20 train/validation/test. *temporal_gap* orders blocks in time, takes the first 60% for training, discards one gap block, and uses the next 20% for validation and the last 20% for test. On these splits we run five settings.

- *LOACO* (grouped_random): a class is held out if it has at least 200 training-pool windows (by majority label) and at least one test window; training and validation exclude every window containing any flow of that class. Every resulting fold has at least 81 held-out test windows.
- *Natural temporal novelty* (temporal_gap): classes that occur in the test period but not in the training period are scored as unseen against test benign windows. Validation is the last 20% of the training period (train-tail validation), because on D1 every temporal validation window contains a natural-novelty flow.
- *Purged block LOACO* (D5): 10-minute blocks are split 60/20/20 within each capture day; training and validation blocks adjacent to a test block, and training blocks adjacent to a validation block, are removed [45]. Here a held-out class also needs at least 50 test windows.
- *In-distribution*: binary detection on the test split of both split schemes.
- *Zero-shot transfer*: models trained on D1–D3 are applied to every other NetFlow-v3 dataset, including D4; every detector reference (E, benign-only fits, Mahalanobis, kNN and CLAD references) comes from the source network, and nothing is fitted on the target.

E. Pre-registration and provenance

Research questions, metrics and acceptance gates were written to a file and hashed before each experiment; every result file records that hash, the code commit and the dataset hashes. Six dated, hashed amendments changed the plan, each before the runs it governs: two because data counts made a frozen rule infeasible (an empty validation split; zero LOACO folds on D5), three that set a new direction after a gate returned STOP, and one that added P(attack) and the family split after the last gate. Four confirmatory gates were evaluated exactly as frozen. The final benchmark reported here is descriptive: it applies no thresholds and reports intervals rather than significance tests.

V. Experimental Setup

Training. The backbone is trained with AdamW (learning rate 10^{-3}, weight decay 10^{-4}), cosine annealing, gradient clipping at 1.0, batch size 256 and 15 epochs, with no per-detector tuning. The checkpoint with the best validation macro-F1 is primary (for CLAD, the best validation AUROC). The final-epoch state of the same training run is also kept and scored as a sensitivity analysis. XGBoost uses subsampling of 1, which makes it deterministic.

Evaluations. LOACO covers D1 families, D2 and D3 (6, 8 and 6 folds); natural novelty covers D1, D3 and D5 (2, 5 and 2 classes); purged LOACO covers D5 (1 fold); in-distribution detection covers D1, D2, D3 and D5 under both splits; and transfer covers 9 source–target pairs.

Metrics and statistics. Seeds are 17, 23, 42, 101 and 202, and a seeding regression test runs before every experiment. For each detector and class we report ROC AUC as the mean over seeds with a Student-t 95% interval (n=5, t=2.776), the seed standard deviation and the fraction of seeds below 0.5. Family means average the family's detectors and the evaluation's classes, with a 95% bootstrap interval that resamples classes and seeds independently (2,000 resamples). In-distribution results additionally report PR-AUC, macro-F1, MCC and TPR at 1% FPR.

Compute and provenance. The 160 final runs used one code commit and a CPU-only cloud container (4 vCPUs, Intel Xeon at 2.10 GHz, no GPU); together they took 4.27 h of wall-clock time (96 s per run on average). The report scripts refuse to run if code commits differ within an experiment. One run interrupted by a container restart was resumed from the same commit.

VI. Results

All values are mean AUC over five seeds; brackets give 95% intervals.

A. C1: Class hold-out is often infeasible on time-ordered captures

Temporal LOACO is infeasible on two of three day-scheduled captures, while natural novelty is available on all three (Table III). Day-scheduled attacks have been observed to make chronological splits degenerate [16], [15]; we treat the same property as a natural zero-day protocol rather than

TABLE III
PROTOCOL FEASIBILITY: FEASIBLE LOACO FOLDS PER SPLIT AND NATURAL-NOVELTY (NN) CLASSES.

Dataset	LOACO, grouped	LOACO, temporal	NN classes
D1	6 (fam.)	0	2
D2	8	8	0
D3	6	0	5
D5	0	0	2
D5 purged block	1 (dos_hulk)		–

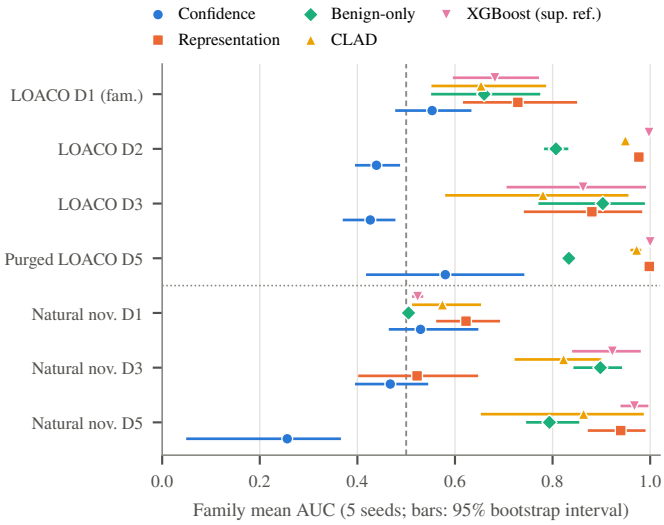


Fig. 2. Family mean AUC per evaluation (markers) with 95% bootstrap intervals over classes and seeds (bars). The leading novelty family changes between LOACO (top four rows) and natural novelty (bottom three); XGBoost (triangles down) is the supervised reference. Dashed line: chance.

discarding it. In D1 and D3 the time-ordered split puts every attack class wholly in the training period or wholly in the test period (D1: 12 and 2 of 14 classes; D3: 4 and 5 of 9), so no class can be held out of a training pool that contains it. D2 interleaves attacks over time and supports LOACO under both splits but has no test-only classes. On D5 the purged 10-minute split kept 125 of 219 blocks and yielded one feasible fold (dos_hulk); the other twelve classes either fell wholly outside the test split or had fewer than 200 training windows (web brute force had 199). **Answer to RQ1:** on realistic, time-ordered captures, natural novelty is often the only zero-day evaluation an operator can run.

B. C2: No detector family dominates

The leading novelty family changes across evaluations (Fig. 2; per-class detail in Fig. 3): representation-distance scores lead in five evaluations and benign-only detectors in two. The largest reversal is on D3. Under LOACO, benign-only and representation scores are close (benign-only minus representation $+0.022$ [$+0.004$, $+0.047$]); under natural novelty, benign-only detectors lead by $+0.376$ [$+0.247$, $+0.500$], and the representation family drops to chance in the mean (0.522 [0.402, 0.647]). Under natural novelty on D1 every family is weak; the best is representation at 0.623 [0.562, 0.692]. CLAD never leads: it trails benign-only detectors on LOACO D3 (-0.123 [-0.277, -0.034]) and natural novelty D3 (-0.075 [-0.130, -0.023]), is below 0.5 on every seed for D3 LOACO ddos (0.372), and beats benign-only detectors on LOACO D2 ($+0.142$ [$+0.119$, $+0.164$]). XGBoost exceeds the leading novelty family on LOACO D2 (0.998 vs 0.977), natural novelty D3 (0.923 vs 0.898) and natural novelty D5 (0.968 vs 0.940), and ties it on purged D5. **Answer to RQ2 (agreement):** the protocols crown different winners, and a dedicated novelty score does not reliably beat a plain supervised classifier.

C. C3: Natural novelty exposes instability that LOACO hides

Every supervised score is less stable under natural novelty than under LOACO (Fig. 4), by 1.7× (MSP) to 17× (kNN). The two seeded benign-only detectors stay below 0.018, below every supervised score's minimum of 0.074; the autoencoder's large ratio comes from a near-zero LOACO baseline and one class (D5 portscan). On D3 natural novelty, kNN has a seed standard deviation of 0.33–0.40 AUC on four of five classes, so the same detector can rank first or last depending on the seed. **Answer to RQ2 (stability):** one training run says little about a supervised detector's zero-day behaviour, and LOACO understates the variation.

D. C4: Confidence-magnitude scores are invalid in binary NIDS

MSP and energy fall below 0.5 wherever the classifier detects unseen attacks confidently; the attack probability does not (Fig. 5). On LOACO D2, MSP is below 0.5 on every seed of all eight classes, while P(attack) reaches 0.997–1.000 on the same models: the classifier detects the held-out attacks, and MSP reads that confidence as normality. The same mechanism appears in-distribution, where MSP ranges from 0.075 (D3) to 0.394 (D1) under grouped splits on D1–D3 while P(attack) is 0.812–0.999. P(attack) falls below 0.5 only on individual classes: D3 LOACO ddos (0.35) and scanning (0.46), and D5 natural-novelty portscan (0.16). Representation distance is a valid alternative in the mean, but on natural novelty D3 its interval includes 0.5. **Answer to RQ3 (validity):** max-softmax and energy should not be used as novelty alarms on binary detectors.

E. C5: Temporal and cross-dataset generalization

Temporal-split effects are dataset-specific (Fig. 6, left). On D1 both models lose about 0.27–0.29 AUC; on D2 neither changes by more than 0.014; on D5 both improve; and only on D3 does the neural backbone degrade (0.664, seed std 0.202) while XGBoost holds 0.978. Across the 108 zero-shot transfer results (12 detectors × 9 source–target pairs; Fig. 6, right) the median AUC is 0.500: 67% fall in [0.3, 0.6], 12% below 0.3 and 21% above 0.6. The highest values come from D1→D3, where CLAD reaches 0.876 and XGBoost 0.832. **Answer to RQ3 (shift):** a detector validated on one network, or under one split, must be revalidated before it is trusted elsewhere.

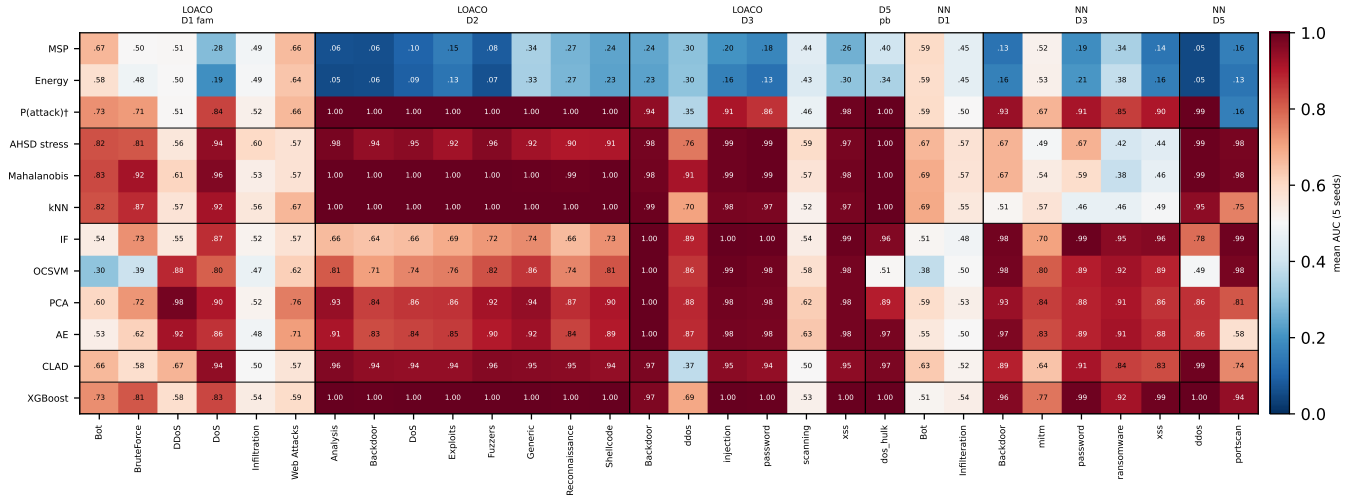


Fig. 3. Mean AUC (five seeds) per detector (rows, grouped by family) and attack class (columns, grouped by protocol and dataset). Red cells are above 0.5 and blue cells below; the leading family changes from one block of columns to the next.

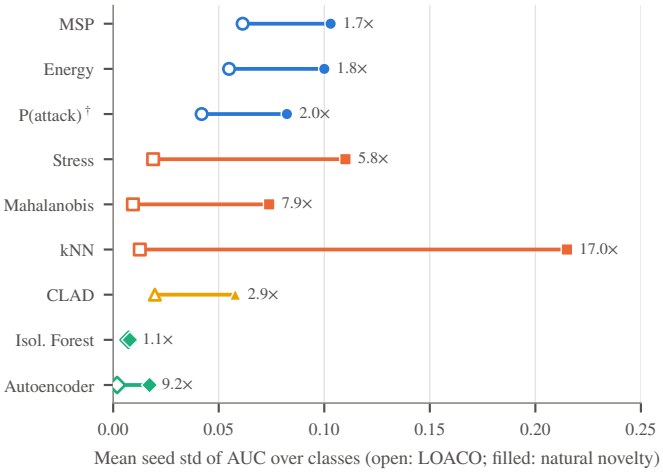


Fig. 4. Seed variability of AUC per detector: open marker LOACO, filled marker natural novelty, labelled with the ratio. Every supervised score becomes 1.7–17× less stable under natural novelty; the seeded benign-only detectors stay near zero. OCSVM, PCA and XGBoost are deterministic and omitted.

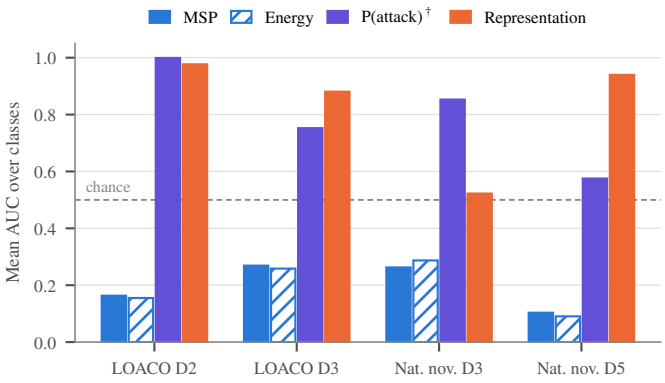


Fig. 5. MSP and energy fall below chance wherever the classifier detects unseen attacks confidently, while the attack probability P(attack) and representation distance stay above chance in the mean.

TABLE IV
PRE-REGISTERED CONFIRMATORY GATES, EVALUATED AS FROZEN. ALL FOUR RETURNED STOP.

Gate	Hypothesis	Frozen rule	Observed
P1	Spectral model predicts per-class detectability	$\rho \geq 0.6$	$\rho = 0.326$
G1	Adaptive equilibrium absorbs persistent attacks	$\rho(\gamma, d') \leq -0.8$	-0.14 / -0.14 / 0.00
G2	Benign drift inverts scores; re-anchoring fixes it	≥5/9 detectors	4, 5, 0 of 9
G3	Benign-only beat supervised only under NN	$B - S \geq 0.15$ / ≤ 0.05	+0.191 / +0.269

F. Checkpoint sensitivity

Using the final-epoch instead of the best-validation checkpoint moves representation and CLAD family means by at most 0.073 AUC and confidence scores by up to 0.205 on D5, whose train-tail validation set holds only 26 attack windows. Only one benign-only contrast changes sign (natural novelty D1, benign-only minus confidence: -0.025 to $+0.032$).

G. C6: Pre-registered negative results

Four confirmatory hypotheses were tested against frozen gates, and all four failed (Table IV). A spectral model of the recurrent filter did not predict per-class detectability (P1). An adaptive equilibrium did not absorb persistent attacks under supervised training; detectability instead peaked at a small adaptation rate (G1). Benign drift with re-anchoring (G2) and benign-only superiority restricted to natural novelty (G3) also failed. G3's supervised family included MSP and energy, which Section VI-D shows are invalid in binary NIDS, so G3 tested a flawed contrast; we keep its verdict as recorded and

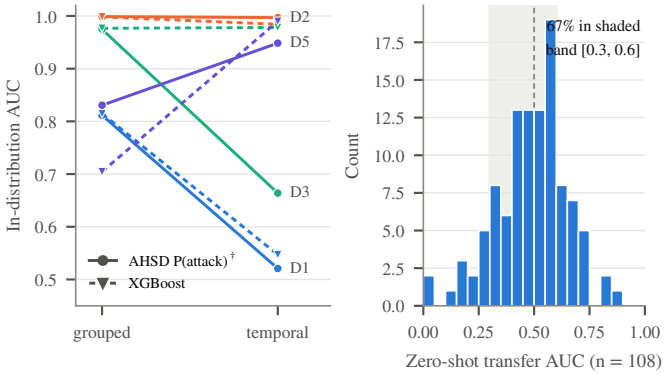


Fig. 6. Left: in-distribution AUC under grouped-random vs temporal splits per dataset (solid: AHSD P(attack); dashed: XGBoost). Only on D3 does the neural backbone degrade while XGBoost holds. Right: the 108 zero-shot transfer AUCs (12 detectors × 9 pairs) cluster around chance.

report the corrected family split above.

VII. Discussion

A. Why the protocols disagree

LOACO removes one class but keeps the rest of the training distribution, including benign traffic from the hours of the test set. Natural novelty changes the attack class, the benign traffic and often the hosts and tools at once. Supervised representations are shaped by the attacks they were trained to separate: when the shift is small they place a held-out class far from benign traffic (representation 0.977 on D2 LOACO), but when benign traffic itself drifts, as on D3, the same scores fall to chance and become seed-dependent. Benign-only detectors do not depend on which attacks were seen, which explains their stability but not their accuracy: they lead on D3 natural novelty yet trail by 0.17 on D2 LOACO. XGBoost's strength agrees with evidence that tree ensembles remain hard to beat on tabular data [44].

B. Implications for security management

For an operator, a NIDS is a security service whose assurance depends on evidence gathered before deployment. Our results show that the usual evidence, a single LOACO run, can select the wrong detector family and understate run-to-run variation by up to 17×. Because retraining is routine in operation, a detector whose zero-day coverage depends on the random seed is hard to manage: two retrainings of the same model can catch different new attack classes. We therefore recommend the following checklist for detector assurance:

1) Evaluate under natural temporal novelty wherever timestamps allow, alongside LOACO, and report when LOACO is infeasible.
2) Do not use max-softmax or energy as novelty alarms on binary detectors; use the attack probability or a representation distance, validated under natural novelty.
3) Include a benign-only detector and a gradient-boosted tree as baselines; one of them matched or beat every dedicated novelty method in five of seven evaluations.
4) Train at least five seeds and report the per-class seed standard deviation, not only the mean.
5) Group flows into time blocks before windowing, deduplicate windows, and purge neighbouring blocks on short captures.
6) Pre-register acceptance criteria and report the ones that fail.

C. Relation to prior findings

Our results agree with Zoppi et al. that detectors trained without attack labels can handle unknown attacks well [21], but only under some protocols; unlike their study and Mesca et al.'s industrial evaluation [17], we find a supervised XGBoost reference at or above the best novelty family in four of seven evaluations, consistent with protocol-dependent rankings [15]. They do not support a general advantage of training with known attacks [22]: CLAD trails benign-only detectors on both D3 evaluations and leads them only on LOACO D2. Reported hold-out results near 0.97 [23] are consistent with our LOACO numbers on D2, and our natural-novelty results suggest that they may not hold under temporal shift. Our finding that time-aware evaluation changes conclusions mirrors earlier evidence from malware detection [3] and intrusion detection [25]. The below-chance MSP results reported for multi-class NIDS [13] are consistent with the binary mechanism we identify in Section VI-D.

D. Limitations and threats to validity

One neural backbone: all supervised scores come from one recurrent backbone, so Fig. 6 is not a claim about neural networks in general. *Small natural-novelty sets:* D1 and D5 contribute two classes each, and the class bootstrap understates their uncertainty; D3 carries most of the evidence. *Thin validation on D5:* 26 attack windows make checkpoint selection fragile for confidence scores. *Design choices:* the 1-hour grouping, any-attack window label, 200-window threshold and 5:1 test ratio were fixed in advance but are not the only reasonable choices. *Post-hoc addition:* P(attack) is labelled and is not confirmatory evidence. *Reimplementation:* CLAD may differ from its authors' code. *Testbed data:* natural novelty follows each capture's schedule, which is closer to deployment than LOACO but is not live traffic.

VIII. Conclusion

How zero-day detection is evaluated decides which detector an operator would deploy. On five public datasets with leakage-controlled splits, 12 detectors and five seeds, class hold-out and natural temporal novelty crowned different winners; natural novelty raised the seed variability of every supervised score by 1.7–17×; max-softmax and energy scores were invalid on binary detectors; and a plain gradient-boosted tree matched or beat the best novelty family in four of seven evaluations. Four pre-registered hypotheses failed and are reported in full. Future work should add neural backbones, more naturally novel classes and live traffic. Code, splits, amendments and a reporting checklist are released at [artifact URL].

REFERENCES

[1] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE S&P*, 2010, pp. 305–316.

[2] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Security*, 2022, pp. 3971–3988.

[3] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. USENIX Security*, 2019, pp. 729–746.

[4] G. Apruzzese, P. Laskov, and J. Schneider, "SoK: Pragmatic assessment of machine learning for network intrusion detection," in *Proc. IEEE EuroS&P*, 2023, pp. 592–614.

[5] M. Husák, D. Manoj, and P. Kumar, "Machine learning in intrusion detection: An operational perspective," in *Proc. CNSM*, 2024, pp. 1–7.

[6] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE SPW*, 2021, pp. 7–12.

[7] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE CNS*, 2022, pp. 254–262.

[8] A. Rosay, F. Carlier, E. Cheval, and P. Leroux, "From CIC-IDS2017 to LYCOS-IDS2017: A corrected dataset for better performance," in *Proc. IEEE/WIC/ACM WI-IAT*, 2021, pp. 570–575.

[9] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. IEEE EuroS&P*, 2024, pp. 658–675.

[10] L. D'hooge, M. Verkerken, B. Volckaert, T. Wauters, and F. De Turck, "Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models," in *Proc. DIMVA*, LNCS, Springer, 2022, pp. 23–41.

[11] U. Sabeel, S. S. Heydari, K. El-Khatib, and K. Elgazzar, "Unknown, atypical and polymorphic network intrusion detection: A systematic survey," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 1, pp. 1190–1212, 2024.

[12] G. Baye, P. Silva, A. Broggi, L. Fiondella, N. D. Bastian, and G. Kul, "Performance analysis of deep-learning based open set recognition algorithms for network intrusion detection systems," in *Proc. IEEE/IFIP NOMS*, 2023, pp. 1–6.

[13] Z. Zhang, Y. Zhang, D. Guo, and M. Song, "A scalable network intrusion detection system towards detecting, discovering, and learning unknown attacks," *Int. J. Mach. Learn. Cybern.*, vol. 12, no. 6, pp. 1649–1665, 2021.

[14] J. Talpini, F. Sartori, and M. Savi, "Enhancing trustworthiness in ML-based network intrusion detection with uncertainty quantification," *J. Reliable Intell. Environ.*, vol. 10, no. 4, pp. 501–520, 2024.

[15] M. Abdessamad and E. B. Tazi, "Feature selection versus feature extraction for IoT intrusion detection under temporal evaluation," *Int. J. Adv. Comput. Sci. Appl.*, vol. 17, no. 8, 2026.

[16] Z. Moczkodan and H. Ragab, "Do transformers actually help intrusion detection? A temporal sequence evaluation on CIC-IDS2017," arXiv:2606.11098, 2026.

[17] S. Mesca, E. Pricop, and G. Stamatescu, "Rigorous evaluation of machine learning intrusion detection for water treatment systems on SWaT network traffic," *Appl. Sci.*, vol. 16, no. 14, Art. 7206, 2026.

[18] S. Nayak, "ThreatFormer-IDS: Robust transformer intrusion detection with zero-day generalization and explainable attribution," arXiv:2603.00185, 2026.

[19] A. Alhalangy and G. E. A. Eltayeb, "Drift-aware continual generative behavioral detection for zero-day cyberattacks: Reproducible temporal evaluation and SIEM-ready alerting," *Information*, vol. 17, no. 9, Art. 852, 2026.

[20] M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, "From zero-shot machine learning to zero-day attack detection," *Int. J. Inf. Secur.*, vol. 22, no. 4, pp. 947–959, 2023.

[21] T. Zoppi, A. Ceccarelli, T. Puccetti, and A. Bondavalli, "Which algorithm can detect unknown attacks? Comparison of supervised, unsupervised and meta-learning algorithms for intrusion detection," *Comput. Secur.*, vol. 127, Art. 103107, 2023.

[22] J. Wilkie, H. Hindy, C. Michie, C. Tachtatzis, J. Irvine, and R. Atkinson, "A novel contrastive loss for zero-day network intrusion detection," *IEEE Trans. Netw. Serv. Manag.*, vol. 23, pp. 2064–2076, 2026.

[23] M. Jodayree, A. Kavoosi Ghafi, S. Amiri, and P. Shaykholeslami, "Explainable zero-day attack detection in IoMT using transformer-based time-series modeling," *Sci. Rep.*, vol. 16, Art. 23252, 2026.

[24] M.-T. Thi and M. Gurusamy, "Multi-dimensional cross-granularity open-set network intrusion detection," *IEEE Trans. Netw. Serv. Manag.*, vol. 23, pp. 4632–4646, 2026.

[25] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "INSOMNIA: Towards concept-drift robustness in network intrusion detection," in *Proc. ACM AISec*, 2021, pp. 111–122.

[26] S. Yang, X. Zheng, J. Li, J. Xu, X. Wang, and E. C. H. Ngai, "ReCDA: Concept drift adaptation with representation enhancement for network intrusion detection," in *Proc. ACM KDD*, 2024, pp. 3818–3828.

[27] X. Zhang *et al.*, "Continual learning with strategic selection and forgetting for network intrusion detection," in *Proc. IEEE INFOCOM*, 2025, pp. 1–10.

[28] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP NOMS*, 2022, pp. 1–9.

[29] S. Layeghy and M. Portmann, "Explainable cross-domain evaluation of ML-based network intrusion detection systems," *Comput. Electr. Eng.*, vol. 108, Art. 108692, 2023.

[30] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144489–144508, 2024.

[31] M. Luay, S. Layeghy, S. Hosseininoorbin, M. Sarhan, N. Moustafa, and M. Portmann, "Temporal analysis of NetFlow datasets for network intrusion detection systems," arXiv:2503.04404, 2025.

[32] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Proc. BDTA 2020*, LNICST 371, Springer, 2021, pp. 117–135.

[33] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108–116.

[34] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. MilCIS*, 2015, pp. 1–6.

[35] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, Art. 102994, 2021.

[36] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, 2019.

[37] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *Proc. ICLR*, 2017.

[38] W. Liu, X. Wang, J. Owens, and Y. Li, "Energy-based out-of-distribution detection," in *Proc. NeurIPS*, 2020, pp. 21464–21475.

[39] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in *Proc. NeurIPS*, 2018.

[40] Y. Sun, Y. Ming, X. Zhu, and Y. Li, "Out-of-distribution detection with deep nearest neighbors," in *Proc. ICML*, 2022, pp. 20827–20840.

[41] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE ICDM*, 2008, pp. 413–422.

[42] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.

[43] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM KDD*, 2016, pp. 785–794.

[44] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. NeurIPS Datasets and Benchmarks*, 2022, pp. 507–520.

[45] M. López de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ, USA: Wiley, 2018, ch. 7.

[46] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.